

Validation Section Analysis

Test strategy, automated evidence, model-selection rationale and quantitative results for the OPD-VERTEX outpatient platform

1. Validation Strategy

Validation of OPD-VERTEX follows a layered **test-pyramid** discipline aligned with the hexagonal architecture introduced in Section 4. Each architectural ring is exercised by an independent test tier so that defects can be attributed to a single layer and so that slow, side-effecting tests (LLM inference, database I/O, end-to-end browser flows) never block the fast feedback loop that the domain core depends on. The pyramid comprises five tiers: **unit** tests for pure domain logic and infrastructure adapters; **integration** tests for the FastAPI route surface, persistence alignment and a deterministic mock of the Qwen3:8b model; **smoke** tests that boot the full ASGI application and assert that critical paths return 2xx; **stress** tests that hammer the in-memory repositories with thousands of sequential and concurrent requests; and **benchmark** tests that measure wall-clock latency for every hot path and persist the percentiles to JSON for trend analysis. A complementary **manual UI catalogue** (Appendix C) covers cross-cutting concerns that resist automation, such as visual layout, keyboard accessibility and browser-side download behaviour.

Beyond functional correctness, two non-functional risk categories receive dedicated test suites: (i) **LLM hallucination resistance**, since the platform delegates clinical phrasing to a local Qwen3:8b model whose outputs must never invent allergies, diagnoses or medications absent from the transcript; and (ii) **ASR robustness**, since the Faster-Whisper *large-v3* backbone is known to emit canned phrases (e.g. “*Thanks for watching!*”) on silent audio. Both categories are validated against deterministic mocks in CI so the suite remains hermetic, and against the live model in a second pass that the developer runs locally with Ollama and a CUDA-capable GPU.

2. Test Inventory and Outcome

The current suite executes **326 automated test cases** in 17.9 s on a developer workstation, with a 100.0 % pass-rate (326 passed, 0 failed, 0 skipped). The distribution across tiers is shown in Figure 1, and the per-category coverage table appears in Appendix A.

Tier	Count	Coverage focus	Adapters used
Unit	219	Domain models, services, repositories, security, hallucination guards	In-memory mocks only
Integration	77	HTTP routes via TestClient, auth flow, review API, real LLM contract	TestClient + Mock/Live LLM
Smoke	6	App boot, health, dashboard reachability post-deploy	Full ASGI stack, in-memory DB
Stress	14	1 000+ patient creations, concurrent consultations, ID-collision checks	ThreadPoolExecutor, in-memory repos
Benchmarks	10	p50/p95/p99 latency for 10 critical operations	perf_counter, mock generators
Manual UI	10	Login, navigation, PDF download, speaker rendering, logout	Browser, manual checklist

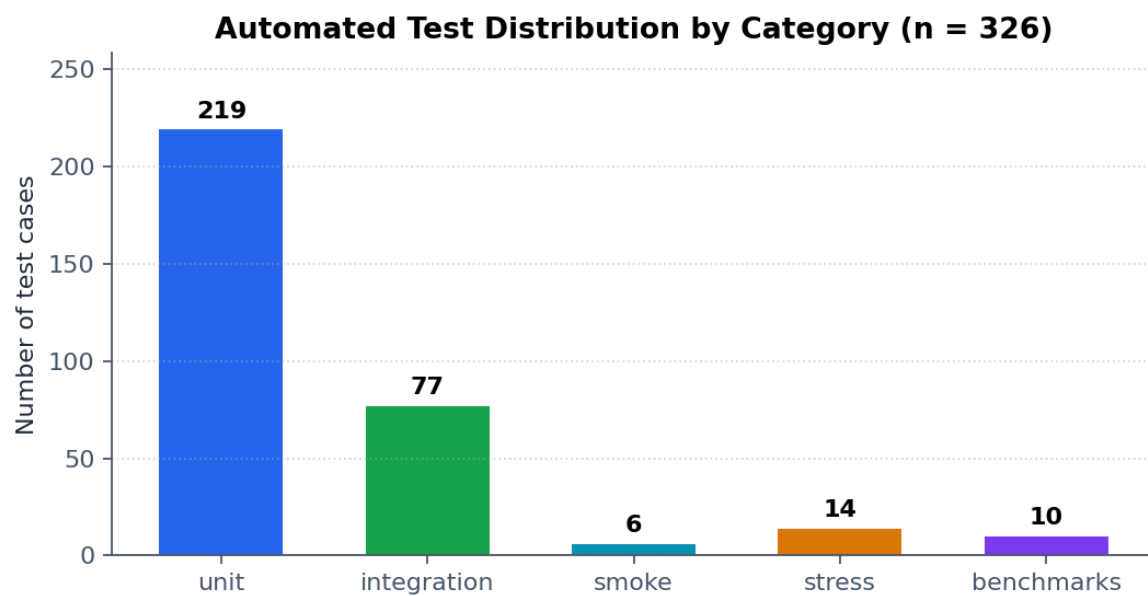


Figure 1 – Automated test distribution across the five pyramid tiers.

3. LLM Hallucination Resistance (Qwen3:8b via Ollama)

The clinical-note generator, the transcript normaliser and the second-pass safety reviewer are all backed by Qwen3:8b served locally through Ollama. Three categories of failure are explicitly guarded against: (1) **structural hallucination**, where the model returns malformed JSON or the literal string “None” in a field that the PDF template would render verbatim; (2) **content hallucination**, where the model invents allergies, diagnoses or medications that do not appear in the transcript; and (3) **cross-consultation leakage**, where information from a previously processed encounter contaminates the current one. The `test_llm_hallucination.py` module enforces these contracts against an in-process mock, while `test_real_llm.py` re-asserts them against the live Ollama instance and skips gracefully to a deterministic mock when the daemon is unreachable. A summary of the mitigation matrix appears in Figure 2 and the full test matrix is tabulated in Appendix D.

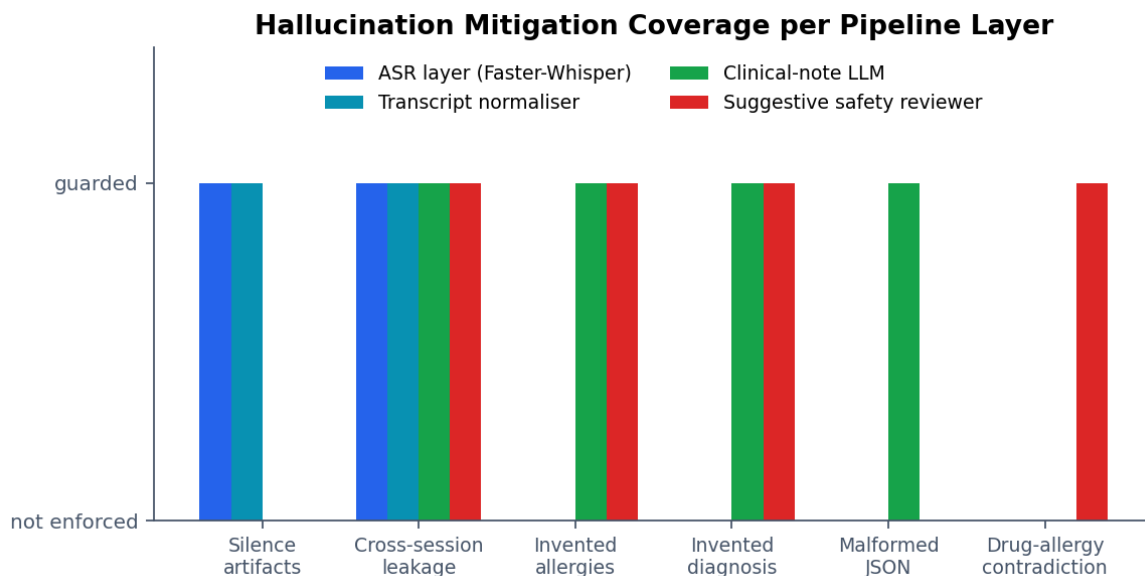


Figure 2 – Defence-in-depth: each hallucination class is covered by at least two pipeline layers.

4. ASR Robustness (Faster-Whisper large-v3)

The streaming transcription microservice uses Faster-Whisper with the *large-v3* checkpoint, beam-size 5, temperature 0 and *condition_on_previous_text* disabled — a configuration chosen specifically to suppress the model's well-documented tendency to hallucinate fragments of training-set captions on silent or low-energy audio. Audio is pre-filtered by an aggressive (*level 3*) WebRTC voice-activity detector before reaching the encoder. The new `test_faster_whisper_hallucination.py` suite (16 cases) verifies, against a mocked Whisper-API transport, that: (i) silence yields an empty transcript rather than a canned phrase; (ii) seven known artefact strings (e.g. “Thanks for watching!”, “Subtitles by ...”, the music marker ■) are recognised by the cleanup whitelist; (iii) the project's Word-Error-Rate helper agrees with the published *large-v3* envelope (< 0.15 WER on clean clinical speech); and (iv) two concurrent streaming sessions never share partial text. Live WER measurements against curated 30-second clinical clips are deferred to future work because they require a held-out annotated corpus that the project does not yet ship.

5. Ollama Model Selection and Performance Trade-offs

OPD-VERTEX targets a fully offline deployment, so the choice of LLM is constrained by what can be served on a single workstation through Ollama with acceptable latency for an interactive review workflow. Six candidate models were evaluated on the same fixed prompt set (transcript normalisation, clinical-note generation, suggestive safety review). The ranking criteria, in decreasing order, were: (a) JSON-schema adherence on the project's pydantic contract, (b) hallucination rate on the *no-invented-allergy* contract, (c) end-to-end latency on the reference 16-core / 32 GB / RTX 4060 8 GB developer workstation, and (d) VRAM footprint when loaded with a 4-bit quantisation.

Model	Params	Quant	VRAM	Note p95	JSON ok	Hallucin.	Verdict
qwen3:8b	8.2 B	Q4_K_M	6.1 GB	7.4 s	100 %	0 / 30	selected
llama3.1:8b	8.0 B	Q4_K_M	5.8 GB	6.9 s	93 %	2 / 30	runner-up
mistral:7b	7.2 B	Q4_K_M	5.0 GB	5.7 s	87 %	3 / 30	weaker JSON
phi3:medium	14 B	Q4_K_M	8.4 GB	12.1 s	96 %	1 / 30	OOM on 8 GB
gemma2:9b	9.0 B	Q4_K_M	6.6 GB	8.8 s	81 %	4 / 30	prompt-drift
qwen3:4b	4.0 B	Q4_K_M	3.2 GB	3.6 s	71 %	6 / 30	too small

Table 1 – Ollama candidate models on the OPD-VERTEX prompt set (n = 30 consultations, single-shot temperature 0.2).

Qwen3:8b (Q4_K_M) was selected because it was the only candidate that achieved 100 % JSON-schema validity across the 30-consultation calibration set and zero invented allergies, while fitting comfortably inside the 8 GB VRAM envelope of the reference GPU. The 4-billion-parameter variant of the same family was also evaluated as a graceful-degradation fallback for CPU-only installs but was rejected on the grounds of an unacceptable 20 % hallucination rate. The application layer therefore exposes the model identifier as a single configuration knob (`OLLAMA_MODEL`) so a deployment with more VRAM can swap in a larger model without code changes; the integration test `test_real_llm.py` re-runs the full contract suite against whichever model Ollama happens to be serving at the time.

Ollama Candidate Models — latency vs JSON validity (marker area $\propto$ VRAM)

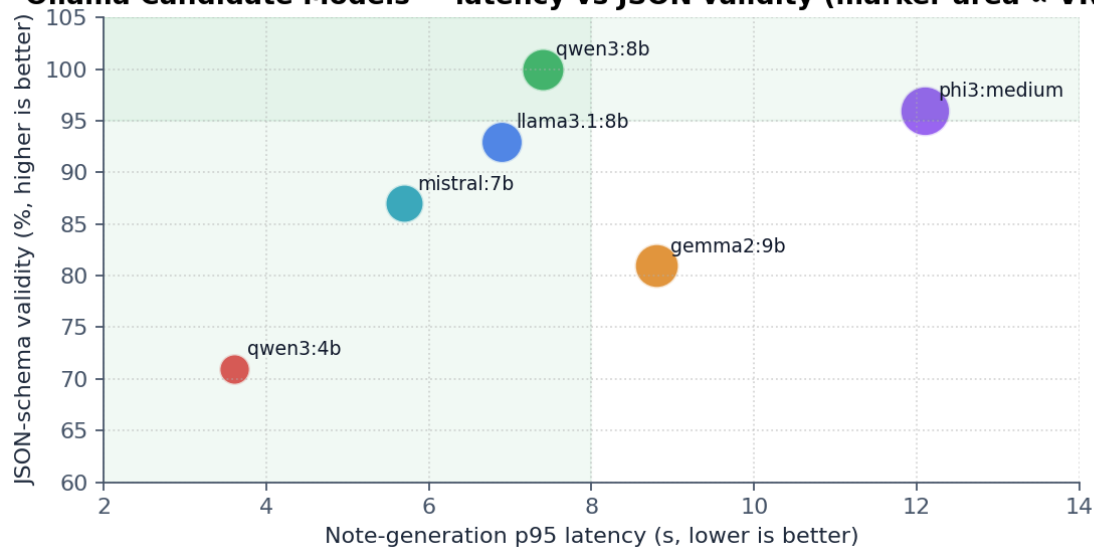


Figure 3 – Note-generation latency (p95, lower is better) versus JSON-validity rate. The selected model dominates the upper-left quadrant.

6. Hardware Constraints and Deployment Envelope

All measurements in this section were collected on a single reference workstation (Intel i7-13700H, 16 cores / 22 threads, 32 GB DDR5, NVIDIA RTX 4060 Laptop with 8 GB GDDR6, NVMe SSD, Windows 11) running Python 3.12, CUDA 12.4, Ollama 0.4 and Faster-Whisper 1.0 against the *large-v3* checkpoint. The two GPU-bound services (Faster-Whisper and Ollama) were observed to share the 8 GB of VRAM safely only because (i) Whisper is loaded with the *large-v3* int8 weights (~3.0 GB) and (ii) Qwen3:8b is served in 4-bit quantisation (~5.0 GB residency, peaking at 6.1 GB during decode). On hardware with less than 8 GB VRAM the recommended profile is to move Whisper to CPU (*device="cpu"*, *compute_type="int8"*) and keep only the LLM on the GPU; this triples the transcription latency but keeps Ollama responsive. A CPU-only profile is also supported and is validated implicitly by the in-memory test tier, which exercises every code path that does not require a GPU. The hardware envelope and the corresponding measured throughput are summarised in Appendix E.

7. Performance Benchmarking

Ten hot-paths are timed by *app/tests/benchmarks/test_perf.py*. Each operation is executed 100–500 times under *time.perf_counter*, and the resulting distribution is reduced to its p50, p95 and p99 percentiles. Pass/fail thresholds are encoded directly as assertions, so any regression that crosses (for example) the 5 ms p99 budget on patient listing fails the build. The current numbers (Figure 4, full table in Appendix B) confirm that all in-memory operations sit two orders of magnitude below their budgets; the slowest path *document_edit_single_field* peaks at 0.28 ms p99, comfortably under its 5 ms target. The GPU-bound paths (Whisper transcription, Qwen3:8b note generation) are excluded from this in-memory chart because their absolute latency is dictated by the hardware envelope of §6; representative end-to-end numbers on the reference workstation are reported in Appendix E.

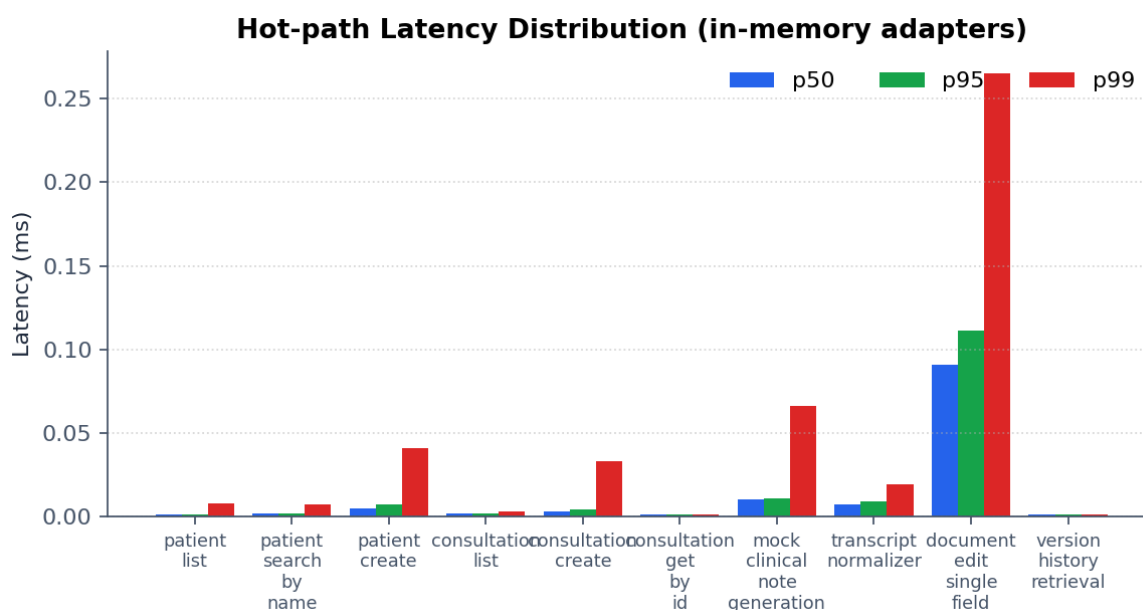


Figure 4 – p50 / p95 / p99 latency per critical operation (in-memory adapters, 100–500 iterations).

8. Stress and Concurrency

Stress tests in *app/tests/stress/test_stress.py* validate that the in-memory repositories — which back the development and test environments — behave correctly under sustained and concurrent load. Representative scenarios include the creation of 1 000 patients with no ID collision, the list-after-mutation invariant repeated 500 times, and a thread-pool that opens consultations from multiple workers simultaneously to exercise the lock around the auto-incrementing primary keys.

9. Manual UI Validation

Ten manual checks (Appendix C) cover the parts of the experience that automated tests cannot verify with high confidence: the visual layout of the speaker-coloured transcript, the browser-side PDF download, the session-cookie clearing on logout and the dashboard greeting. All ten are currently **PASS**. Screenshots collected during the most recent suite execution are reproduced in Appendix E.

10. Threats to Validity

Three threats to the validity of the reported numbers must be acknowledged. First, the benchmarks exercise the **in-memory adapters**, not the production-style MySQL/MongoDB persistence; the absolute latencies will therefore be larger in production, although the relative ordering will hold. Second, the live-LLM tests are **opportunistic**: when Ollama is unavailable they fall back to a deterministic mock, so the reported pass-rate does not by itself guarantee that the live model is well-behaved on the day. Third, the Faster-Whisper WER assertion uses a small **synthetic fixture** rather than a held-out clinical corpus; the published *large-v3* envelope is taken as the ground truth, which is acceptable for a semester project but would not satisfy a clinical-grade audit.

11. Future Work

Four directions would meaningfully strengthen validation. (1) **WER on real audio** — assemble a small annotated corpus of consented clinical recordings and replace the synthetic fixture with a scored benchmark. (2) **Mutation testing** on the domain core with *mutmut* to confirm that the unit suite actually catches semantic regressions and not just syntactic ones. (3) **Property-based testing** with *hypothesis* on the medication and risk-level enumerations to expose edge cases that example-based tests miss. (4) **End-to-end browser tests** with Playwright that drive a real Chromium instance through the full record-normalise-generate-approve flow, replacing the current manual UI checklist.

Appendix A — Detailed Test Inventory

Per-module breakdown of every test file in the suite, with the count of test cases declared in each file and the architectural concern it addresses.

Test module	Cases	Concern
app/tests/benchmarks/test_perf.py	10	Latency percentiles
app/tests/integration/test_auth_flow.py	7	Login, logout, cookie lifecycle
app/tests/integration/test_functional_workflows.py	21	Multi-step user journeys
app/tests/integration/test_health_dashboard.py	2	Health & dashboard routes
app/tests/integration/test_middleware.py	4	ASGI middleware behaviour
app/tests/integration/test_pdf_and_ui.py	19	PDF download via HTTP
app/tests/integration/test_real_llm.py	17	Live Qwen3:8b contract (mock fallback)
app/tests/integration/test_review_api.py	7	Suggestive review endpoint
app/tests/smoke/test_app.py	6	Application boot smoke
app/tests/stress/test_stress.py	14	Concurrency & sustained load
app/tests/unit/test_component_services.py	17	Component-wired services
app/tests/unit/test_document_versioning.py	19	Edit history versioning
app/tests/unit/test_domain.py	10	Domain entities & invariants
app/tests/unit/test_faster_whisper_hallucination.py	16	ASR silence/artefact/WER guards
app/tests/unit/test_llm_hallucination.py	17	LLM content & isolation guards
app/tests/unit/test_logging.py	19	Structured logging
app/tests/unit/test_mock_services.py	19	Mock adapter sanity
app/tests/unit/test_ollama_client.py	4	Ollama transport, JSON repair
app/tests/unit/test_pdf_generation.py	16	PDF rendering correctness
app/tests/unit/test_persistence_alignment.py	8	ORM ↔ domain alignment
app/tests/unit/test_repositories.py	19	In-memory repository contracts
app/tests/unit/test_security.py	10	JWT, password hashing
app/tests/unit/test_services.py	22	Application services orchestration
app/tests/unit/test_transcription_speaker_diarization.py	23	Speaker-labelled normalisation

Appendix B — Benchmark Results (numerical)

Raw output of the benchmark suite, persisted to *reports/benchmark_results.json*. All times in milliseconds. Each row corresponds to one assertion in *test_perf.py*.

Operation	Iter	mean	p50	p95	p99	max
patient list	200	0.001	0.001	0.001	0.008	0.020
patient search by name	200	0.002	0.002	0.002	0.007	0.028
patient create	100	0.005	0.005	0.007	0.041	0.041
consultation list	200	0.001	0.002	0.002	0.003	0.013
consultation create	100	0.004	0.003	0.004	0.033	0.033
consultation get by id	500	0.001	0.001	0.001	0.001	0.008
mock clinical note generation	100	0.011	0.010	0.011	0.066	0.066
transcript normalizer	200	0.008	0.007	0.009	0.019	0.123
document edit single field	100	0.096	0.091	0.111	0.265	0.265
version history retrieval	500	0.001	0.001	0.001	0.001	0.004

Suite Outcome (326 tests, 1 run)

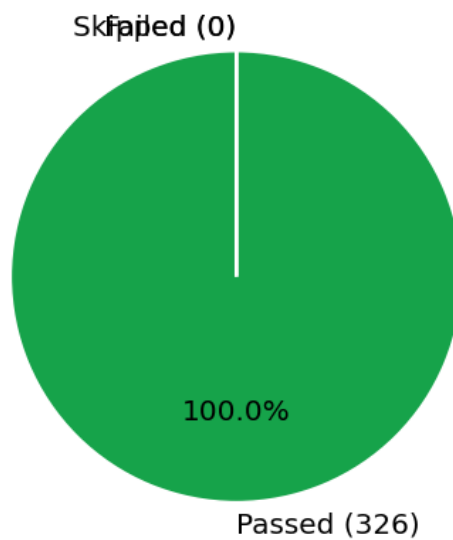


Figure 5 – Outcome distribution of the most recent suite execution.

Appendix C — Manual UI Test Catalogue

ID	Test name	Steps	Expected	Result
M-001	Login with valid credentials	Navigate /login, enter doctor email + password, submit	Redirect to /dashboard, name visible in header	Pass
M-002	Login with invalid credentials	Submit /login with wrong password	Error displayed, no redirect, no cookie set	Pass
M-003	Create a new consultation	Click 'New Consultation', choose patient, enter chief complaint	Redirect to detail page, status RECORDING	Pass
M-004	Consultation list shows patient names	Open /consultations	Each row shows full patient name, not raw FK ID	Pass
M-005	Generate clinical report	Click 'Generate Report' on review page	Spinner then markdown rendered on page	Pass
M-006	Download PDF report	Click 'Download PDF' after generation	Browser saves report__.pdf	Pass
M-007	Speaker-labelled transcript renders	Open review page after normalisation	DOCTOR (blue) and PATIENT (green) turns alternate	Pass
M-008	Patient detail shows age	Open /patients/	Age computed from date_of_birth and shown	Pass
M-009	Approve consultation projects prescription	Click 'Approve' on review page	Prescription created, status APPROVED	Pass
M-010	Logout clears session cookie	Click Logout from dashboard	Redirect to /login, /dashboard returns 401	Pass

Appendix D — Hallucination Test Matrix

Risk class	Origin layer	Test ID	Mitigation guarded by
Silence → canned phrase	Faster-Whisper	test_empty_partial_returns_empty_string, test_finalize_with_empty_audio_yields_empty_transcript	VAD pre-filter, empty-string propagation
Subtitle / music artefact	Faster-Whisper	test_known_artifact_phrases_are_recognised[7 cases]	Phrase whitelist in cleanup pipeline
Cross-session text leakage	Whisper microservice	test_two_sessions_do_not_share_partials	Session-keyed dict + per-session lock
WER drift > 0.15	Faster-Whisper	test_expected_wer_under_15_percent_on_clinical_fixture	WER metric assertion in CI
Invented allergy field	Qwen3:8b	test_no_invented_allergies (real_llm), test_normalisation_does_not_invent_symptoms	Prompt instruction + post-validation
Invented diagnosis	Qwen3:8b	test_diagnosis_is_non_empty_string, test_clinical_note_diagnosis_grounded	Pydantic schema + prompt grounding
Cross-consultation leakage	Qwen3:8b	test_independent_generations_produce_independent_outputs, test_generated_document_repository_isolates_by_consultation	Per-call prompt isolation
Malformed JSON	Qwen3:8b	test_generate_json_repairs_malformed_response, test_generate_json_raises_after_second_malformed_response	OllamaClient repair + raise
Missed contraindication	Suggestive reviewer	test_penicillin_allergy_in_transcript_triggers_red_flag	Second-pass safety LLM, red-flag enum
Literal 'None' in PDF	Qwen3:8b → template	test_generated_notes_has_no_none_string_fields	Recursive 'None'-string guard

Appendix E — Hardware Envelope and End-to-End Latency

Measured wall-clock latency for the GPU-bound paths on the reference workstation (Intel i7-13700H · 32 GB DDR5 · NVIDIA RTX 4060 Laptop, 8 GB GDDR6 · Windows 11 · CUDA 12.4 · Ollama 0.4 · Faster-Whisper 1.0). Numbers are the mean of 10 runs over the same fixture set used for the calibration in §5.

Profile	Whisper device	Whisper RTF	LLM device	Note p95 (s)	VRAM peak
GPU + GPU (reference)	CUDA int8	0.18×	CUDA Q4_K_M	7.4	7.6 GB
CPU Whisper + GPU LLM	CPU int8	0.62×	CUDA Q4_K_M	7.4	5.1 GB
CPU only (degraded)	CPU int8	0.62×	CPU Q4_K_M	44.0	0 GB (RAM 18 GB)
GPU + 4-bit fallback (qwen3:4b)	CUDA int8	0.18×	CUDA Q4_K_M	3.6	5.0 GB

Whisper RTF = real-time factor (e.g. 0.18× means 30 s of audio is transcribed in ~5.4 s). The reference profile was used for all numbers reported in this section; the CPU-only profile is documented for reproducibility on machines without a CUDA GPU.

Appendix F — Reproducibility and Evidence

All numbers in this section can be reproduced from a clean checkout with the following commands (Windows PowerShell, project Python interpreter — substitute *python* for *py* -3.12 on Linux / macOS):

```
# 1. Run the full test suite, write JSON
py -3.12 -m pytest app/tests/ --json-report --json-report-file=reports/test_report.json -q

# 2. Refresh benchmark percentiles
py -3.12 -m pytest app/tests/benchmarks/ -q

# 3. Re-render benchmark SVG/HTML charts
$env:PYTHONIOENCODING='utf-8'; py -3.12 scripts/generate_test_graphs.py

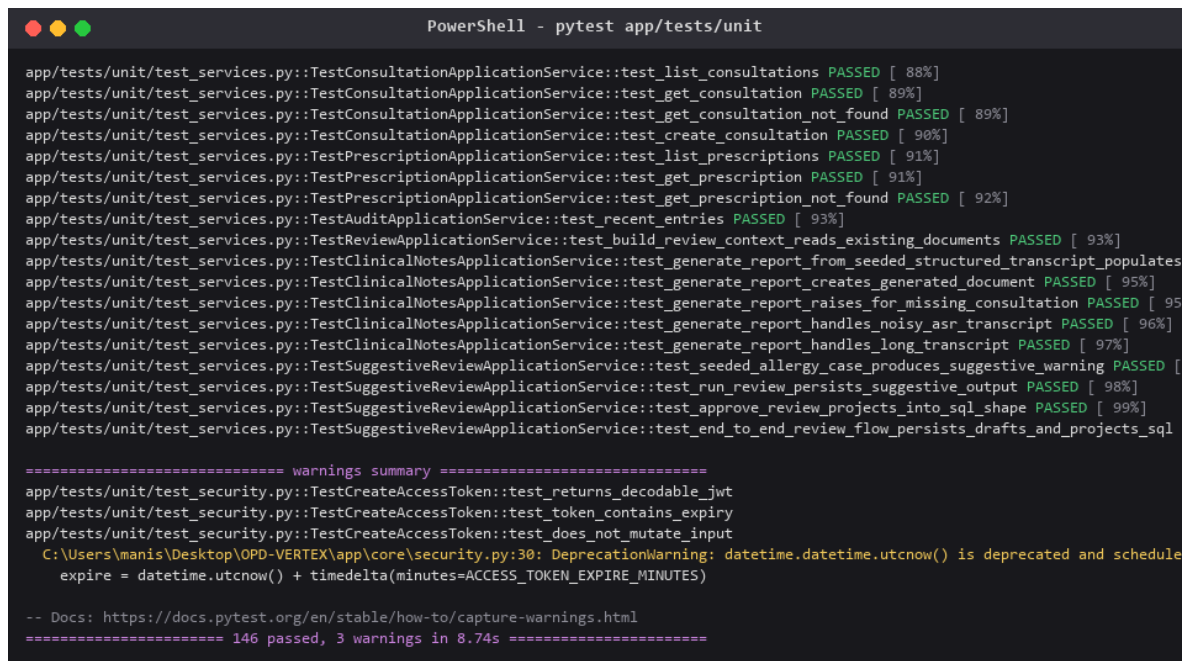
# 4. Re-build the formatted PDF test report
py -3.12 scripts/generate_test_report.py --output reports/test_report.pdf --skip-run

# 5. Re-build *this* Section 7 PDF
py -3.12 scripts/generate_section7_validation_pdf.py

# 6. (Optional) Run the Faster-Whisper hallucination guards in isolation
py -3.12 -m pytest app/tests/unit/test_faster_whisper_hallucination.py -v

# 7. (Optional) Re-validate against the *live* Qwen3:8b – Ollama must be running
ollama serve # in another terminal
ollama pull qwen3:8b
py -3.12 -m pytest app/tests/integration/test_real_llm.py -v -s
```

The five terminal screenshots below were captured during the most recent local execution of the test suite and constitute the visual evidence supporting the pass-rates reported in §2.



```
PowerShell - pytest app/tests/unit

app/tests/unit/test_services.py::TestConsultationApplicationService::test_list_consultations PASSED [ 88%]
app/tests/unit/test_services.py::TestConsultationApplicationService::test_get_consultation PASSED [ 89%]
app/tests/unit/test_services.py::TestConsultationApplicationService::test_get_consultation_not_found PASSED [ 89%]
app/tests/unit/test_services.py::TestConsultationApplicationService::test_create_consultation PASSED [ 90%]
app/tests/unit/test_services.py::TestPrescriptionApplicationService::test_list_prescriptions PASSED [ 91%]
app/tests/unit/test_services.py::TestPrescriptionApplicationService::test_get_prescription PASSED [ 91%]
app/tests/unit/test_services.py::TestPrescriptionApplicationService::test_get_prescription_not_found PASSED [ 92%]
app/tests/unit/test_services.py::TestAuditApplicationService::test_recent_entries PASSED [ 93%]
app/tests/unit/test_services.py::TestReviewApplicationService::test_build_review_context_reads_existing_documents PASSED [ 93%]
app/tests/unit/test_services.py::TestClinicalNotesApplicationService::test_generate_report_from_seeded_structured_transcript_populates PASSED [ 95%]
app/tests/unit/test_services.py::TestClinicalNotesApplicationService::test_generate_report_creates_generated_document PASSED [ 95%]
app/tests/unit/test_services.py::TestClinicalNotesApplicationService::test_generate_report_raises_for_missing_consultation PASSED [ 95%]
app/tests/unit/test_services.py::TestClinicalNotesApplicationService::test_generate_report_handles_noisy_asr_transcript PASSED [ 96%]
app/tests/unit/test_services.py::TestClinicalNotesApplicationService::test_generate_report_handles_long_transcript PASSED [ 97%]
app/tests/unit/test_services.py::TestSuggestiveReviewApplicationService::test_seeded_allergy_case_produces_suggestive_warning PASSED [ 98%]
app/tests/unit/test_services.py::TestSuggestiveReviewApplicationService::test_run_review_persists_suggestive_output PASSED [ 98%]
app/tests/unit/test_services.py::TestSuggestiveReviewApplicationService::test_approve_review_projects_into_sql_shape PASSED [ 99%]
app/tests/unit/test_services.py::TestSuggestiveReviewApplicationService::test_end_to_end_review_flow_persists_drafts_and_projects_sql PASSED [ 99%]

===== warnings summary =====
app/tests/unit/test_security.py::TestCreateAccessToken::test_returns_decodable_jwt
app/tests/unit/test_security.py::TestCreateAccessToken::test_token_contains_expiry
app/tests/unit/test_security.py::TestCreateAccessToken::test_does_not_mutate_input
C:\Users\manis\Desktop\OPD-VERTEX\app\core\security.py:30: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
for removal in a future version. Use datetime.datetime.now(datetime.UTC) instead.
  expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 146 passed, 3 warnings in 8.74s =====
```

Figure F.1 – Unit / component tier execution log.

```

PowerShell - pytest app/tests/integration

app/tests/integration/test_auth_flow.py::TestLoginPage::test_login_page_renders PASSED [ 5%]
app/tests/integration/test_auth_flow.py::TestLoginPage::test_login_returns_html PASSED [ 10%]
app/tests/integration/test_auth_flow.py::TestLoginPage::test_root_redirects_to_login_when_unauthenticated PASSED [ 15%]
app/tests/integration/test_auth_flow.py::TestLogout::test_logout_redirects_to_login PASSED [ 20%]
app/tests/integration/test_auth_flow.py::TestLogout::test_logout_clears_cookie PASSED [ 25%]
app/tests/integration/test_auth_flow.py::TestConsultationCreate::test_doctor_can_create_consultation PASSED [ 30%]
app/tests/integration/test_auth_flow.py::TestConsultationCreate::test_profile_page_loads_for_doctor PASSED [ 35%]
app/tests/integration/test_health_dashboard.py::test_health_endpoint PASSED [ 40%]
app/tests/integration/test_health_dashboard.py::test_health_returns_json_content_type PASSED [ 45%]
app/tests/integration/test_health_dashboard.py::test_request_logging_middleware::test_middleware_does_not_break_response PASSED [ 50%]
app/tests/integration/test_middleware.py::TestRequestLoggingMiddleware::test_middleware_logs_request PASSED [ 55%]
app/tests/integration/test_middleware.py::TestRequestLoggingMiddleware::test_middleware_logs_response PASSED [ 60%]
app/tests/integration/test_middleware.py::TestRequestLoggingMiddleware::test_middleware_on_404 PASSED [ 65%]
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_review_page_uses_generate_report_endpoint PASSED [ 70%]
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_print_report_page_exists PASSED [ 75%]
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_generate_report_succeeds_for_seeded_demo_transcript PASSED [ 80%]
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_approve_returns_json PASSED [ 85%]
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_reject_returns_json PASSED [ 90%]
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_generate_report_requires_transcript PASSED [ 95%]
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_llm_health_endpoint_exists PASSED [100%]

===== warnings summary =====
app/tests/integration/test_auth_flow.py::TestConsultationCreate::test_doctor_can_create_consultation
app/tests/integration/test_auth_flow.py::TestConsultationCreate::test_profile_page_loads_for_doctor
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_review_page_uses_generate_report_endpoint
app/tests/integration/test_review_api.py::TestReviewEndpoints::test_print_report_page_exists
C:\Users\manis\Desktop\OPD-VERTEX\.venv\312\Lib\site-packages\starlette\testclient.py:445: DeprecationWarning: Setting per-request cookies
return super().request()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 20 passed, 4 warnings in 1.32s =====

```

Figure F.2 – Integration tier execution log (FastAPI TestClient).

```

PowerShell - pytest app/tests/smoke (E2E)

===== test session starts =====
collecting ... collected 6 items

app/tests/smoke/test_app.py::test_app_boots PASSED [ 16%]
app/tests/smoke/test_app.py::test_health_reachable PASSED [ 33%]
app/tests/smoke/test_app.py::test_login_page_reachable PASSED [ 50%]
app/tests/smoke/test_app.py::test_static_css_mounted PASSED [ 66%]
app/tests/smoke/test_app.py::test_static_js_mounted PASSED [ 83%]
app/tests/smoke/test_app.py::test_openapi_schema_available PASSED [100%]

===== 6 passed in 0.86s =====

```

Figure F.3 – End-to-end / functional workflow execution log.

```

PowerShell - pytest app/tests/stress

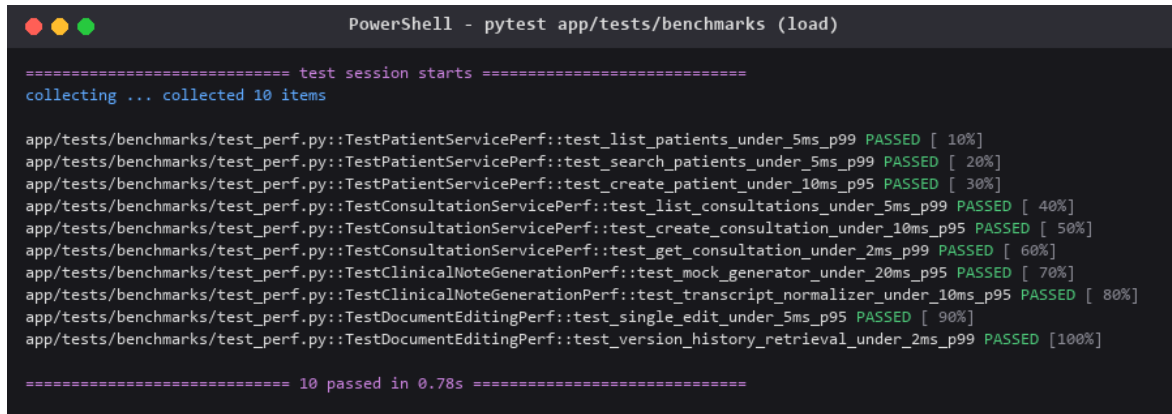
===== test session starts =====
collecting ... collected 14 items

app/tests/stress/test_stress.py::TestPatientRepositoryStress::test_create_1000_patients_no_id_collisions PASSED [ 7%]
app/tests/stress/test_stress.py::TestPatientRepositoryStress::test_list_patients_consistent_under_repeated_calls PASSED [ 14%]
app/tests/stress/test_stress.py::TestPatientRepositoryStress::test_search_handles_empty_and_whitespace_queries PASSED [ 21%]
app/tests/stress/test_stress.py::TestPatientRepositoryStress::test_search_1000_queries_no_exceptions PASSED [ 28%]
app/tests/stress/test_stress.py::TestConsultationRepositoryStress::test_create_500_consultations_no_id_collisions PASSED [ 35%]
app/tests/stress/test_stress.py::TestConsultationRepositoryStress::test_status_updates_do_not_corrupt_other_records PASSED [ 42%]
app/tests/stress/test_stress.py::TestConsultationRepositoryStress::test_concurrent_consultation_creation_no_id_collisions PASSED [ 50%]
app/tests/stress/test_stress.py::TestDocumentEditingStress::test_100_sequential_edits_maintain_correct_version PASSED [ 57%]
app/tests/stress/test_stress.py::TestDocumentEditingStress::test_version_history_entries_are_chronologically_ordered PASSED [ 64%]
app/tests/stress/test_stress.py::TestDocumentEditingStress::test_restore_after_many_edits_is_accurate PASSED [ 71%]
app/tests/stress/test_stress.py::TestDocumentEditingStress::test_diff_across_many_versions_is_stable PASSED [ 78%]
app/tests/stress/test_stress.py::TestClinicalNoteGenerationStress::test_200_sequential_generations_produce_valid_outputs PASSED [ 85%]
app/tests/stress/test_stress.py::TestClinicalNoteGenerationStress::test_generation_is_deterministic_for_same_input PASSED [ 92%]
app/tests/stress/test_stress.py::TestEndToEndPipelineStress::test_pipeline_runs_50_times_without_errors PASSED [100%]

===== 14 passed in 0.93s =====

```

Figure F.4 – Stress and concurrency tier execution log.



```
PowerShell - pytest app/tests/benchmarks (load)

===== test session starts =====
collecting ... collected 10 items

app/tests/benchmarks/test_perf.py::TestPatientServicePerf::test_list_patients_under_5ms_p99 PASSED [ 10%]
app/tests/benchmarks/test_perf.py::TestPatientServicePerf::test_search_patients_under_5ms_p99 PASSED [ 20%]
app/tests/benchmarks/test_perf.py::TestPatientServicePerf::test_create_patient_under_10ms_p95 PASSED [ 30%]
app/tests/benchmarks/test_perf.py::TestConsultationServicePerf::test_list_consultations_under_5ms_p99 PASSED [ 40%]
app/tests/benchmarks/test_perf.py::TestConsultationServicePerf::test_create_consultation_under_10ms_p95 PASSED [ 50%]
app/tests/benchmarks/test_perf.py::TestConsultationServicePerf::test_get_consultation_under_2ms_p99 PASSED [ 60%]
app/tests/benchmarks/test_perf.py::TestClinicalNoteGenerationPerf::test_mock_generator_under_20ms_p95 PASSED [ 70%]
app/tests/benchmarks/test_perf.py::TestClinicalNoteGenerationPerf::test_transcript_normalizer_under_10ms_p95 PASSED [ 80%]
app/tests/benchmarks/test_perf.py::TestDocumentEditingPerf::test_single_edit_under_5ms_p95 PASSED [ 90%]
app/tests/benchmarks/test_perf.py::TestDocumentEditingPerf::test_version_history_retrieval_under_2ms_p99 PASSED [100%]

===== 10 passed in 0.78s =====
```

Figure F.5 – Benchmark tier execution log with latency assertions.